

Estructura de Datos	TRABAJO PRÁCTICO N° 1 - Ciclo 2026 Tema: POO
INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY	

El trabajo práctico permite que el estudiante conozca y aplique conceptos básicos de la programación orientada a objetos.

Para la resolución de los ejercicios se requiere del desarrollo de clases sencillas y la manipulación de objetos de esas clases.

Ejercicios propuestos

- 1) Diseñar una clase llamada Cilindro que representa una figura geométrica tridimensional. Esta clase debe tener los atributos radio (de la base) y altura. La clase Cilindro debe incluir:
 - a) Constructor predeterminado que inicialice ambos valores en 1.0.
 - b) Un constructor con argumentos para establecer el radio y la altura del cilindro.
 - c) Un método que devuelva el volumen del cilindro (Fórmula: $\pi * \text{radio}^2 * \text{altura}$).
 - d) Un método que devuelva el área de la superficie total (Fórmula: $2 * \pi * \text{radio} * (\text{radio} + \text{altura})$).

En el programa principal (main) se debe:

- e) Crear una única instancia de la clase Cilindro.
- f) Solicitar al usuario que ingrese el radio y la altura verificando que los valores sean estrictamente positivos (mayores a cero).
- g) Utilizar los métodos de la clase Cilindro para establecer estas dimensiones.
- h) Calcular y mostrar el volumen y el área de la superficie del cilindro creado, formateando los decimales a dos cifras.

Indicaciones:

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

Preguntas sobre el problema

- a) ¿Cuáles son las entradas y salidas del programa? ¿Qué información debe mantenerse almacenada en el objeto durante toda la ejecución y qué información es sólo temporal?
- b) ¿Qué ocurre si el usuario intenta ingresar un radio o altura igual a cero o a un valor negativo? ¿Cómo garantiza su código que el objeto Cilindro nunca contenga dimensiones inválidas?
- c) Compruebe el comportamiento de un objeto cilindro cuando se usan: valores positivos, valores que intenten vulnerar la validación (por ej. valor negativo) y cuando el radio y la altura son iguales.

Estructura de Datos	TRABAJO PRÁCTICO N° 1 - Ciclo 2026 Tema: POO
INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY	

- d) ¿Qué ventajas tiene implementar los cálculos de volumen y área como métodos en la clase Cilindro en lugar de hacer las operaciones matemáticas directamente en el método main?
- 2) Crear una clase llamada CuentaBancaria que modele una cuenta bancaria. Esta clase debe contener los siguientes atributos: titular, saldo y tipoCuenta (ahorro o corriente). La clase CuentaBancaria debe implementar:
- Constructor predeterminado que inicializa el titular con "Invitado", saldo en 0 y tipoCuenta en "ahorro".
 - Constructor con argumentos que permite establecer el titular, saldo inicial y tipo de cuenta.
 - Un método depositar(double monto) que simule la adición de fondos a la cuenta. Debe validar que el monto sea positivo.
 - Un método retirar(double monto) que simule la extracción de dinero. Debe disminuir el saldo y retornar true si la operación es exitosa. Si el monto excede el saldo disponible, no debe realizar la extracción, debe mostrar un mensaje de error por consola y retornar false.
 - El método toString() que imprima en consola el titular, tipo de cuenta y saldo actual formateado.

En el programa principal (main) se debe:

- Crear una instancia de CuentaBancaria.
- Inicializar la cuenta con algunos valores (ej. "Juan Perez", 50000.0, "corriente").
- Permitir al usuario, a través de un menú sencillo, realizar operaciones de depósito o retiro para esta cuenta, mostrando los detalles actualizados después de cada operación. El menú debe repetirse hasta que el usuario decida salir.

Indicaciones:

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

Preguntas sobre el problema

- ¿Por qué es conveniente que el atributo saldo sea privado y no se pueda modificar directamente desde el método main (ej. cuenta.saldo = 1000000;)? ¿Qué garantiza el uso de los métodos depositar y retirar?
- Un compañero propone que el método retirar sea de tipo void (sin retorno) y que solo muestre un mensaje de error si falla. Otro propone que retorne un boolean como pide el enunciado. ¿Qué ventajas y desventajas presenta cada alternativa? ¿Cuál solución permite que se tomen mejores decisiones?

Estructura de Datos	TRABAJO PRÁCTICO N° 1 - Ciclo 2026 Tema: POO
INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY	

c) ¿Qué sucede si el usuario intenta retirar exactamente el mismo monto que tiene de saldo disponible? ¿Y si intenta depositar un monto negativo?

d) El banco ahora pide agregar un límite de descubierto para las cuentas "corriente". Es decir, si es de tipo corriente, el saldo puede llegar a ser negativo hasta un límite (ej. -50000). ¿Qué método de la clase debería modificar? y ¿qué nueva validación debería agregar?

3) Desarrollar un programa que simule la gestión de una reserva de hotel con fecha de check-in y fecha de check-out, incorporando funcionalidades de verificación y modificación.

El programa debe incluir:

- Una Clase Reserva con los atributos: codigoReserva (String único), nombreHuesped, fechaCheckIn y fechaCheckOut.
- Constructor predeterminado que inicializa codigoReserva en "000", nombreHuesped en "Por confirmar", fechaCheckIn con la fecha actual y fechaCheckOut igual a un día después de la fecha actual.
- Constructor parametrizado que permite establecer todos los atributos mediante argumentos.
- Método toString() para retornar una representación textual completa y legible del objeto Reserva.
- Método auxiliar para validar fechas que comprueba si las fechas de check-in y check-out son válidas (la fecha de check-in debe ser estrictamente anterior a la fecha de check-out). Si las fechas no son válidas, se ajustan automáticamente asignando la fecha actual al check-in y el día siguiente al check-out.
- Un método estaActiva(LocalDate fechaConsulta) para determinar si la reserva está vigente en una fecha dada por el usuario. Debe retornar true si la fechaConsulta es igual o posterior a fechaCheckIn y estrictamente anterior a fechaCheckOut. De lo contrario, retorna false.
- Método calcularDiasRestantesOTranscurridos(LocalDate fechaConsulta). Si la fecha de consulta es anterior al check-in, debe retornar un valor positivo que represente los días faltantes para el check-in. Si la fecha de consulta está dentro de la estadía, retornar 0. Si la fecha de consulta es posterior al check-out, debe retornar un valor negativo que represente los días que han transcurrido desde el check-out hasta la fechaConsulta.
- Método simularProrroga(int cantidadDias) que calcula y retorna la nueva fecha de check-out si la estadía se extendiera por una cantidad específica de días a partir de la fecha de check-out actual.

En el programa principal (main) se debe:

- Solicitar al usuario los datos para crear al menos dos objetos Reserva diferentes, por ejemplo: una reserva futura y una reserva ya finalizada. Se deben realizar las validaciones correspondientes.
- Después de crear las reservas, el programa debe solicitar al usuario una "fecha de consulta" (día, mes, año) para verificar el estado de las mismas. Esta fecha también debe ser validada.

Estructura de Datos	TRABAJO PRÁCTICO N° 1 - Ciclo 2026 Tema: POO
INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY	

Para cada objeto Reserva creado, el programa debe:

- k) Mostrar sus detalles utilizando el método toString().
- l) Llamar al método estaActiva() pasándole la "fecha de consulta" y mostrar el resultado.
- m) Llamar al método calcularDiasRestantesOTranscurridos() pasándole la misma "fecha de consulta" y mostrar la cantidad de días.
- n) Solicitar al usuario una cantidad de días para simular una prórroga, llamar al método simularProrroga() y mostrar la nueva fecha de check-out calculada.

Indicaciones:

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código controle los problemas que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

Preguntas sobre el problema

- a) ¿Por qué se pide que la validación de fechas sea un "método auxiliar" dentro de la clase Reserva? ¿Qué ocurriría si esa validación se hiciera en el método main antes de llamar al constructor?
 - b) ¿Qué sucede si el usuario ingresa la misma fecha para check-in y check-out? Según la regla del inciso (e) ¿qué debería ocurrir con el objeto?
 - c) Si una reserva tiene fechaCheckIn = 2025-05-01 y fechaCheckOut = 2025-05-05, y el usuario ingresa fechaConsulta = 2025-05-03. Sin ejecutar el programa: ¿El método estaActiva retorna true o false? ¿Qué valor exacto retorna calcularDiasRestantesOTranscurridos?
 - d) Un compañero escribe el método simularProrroga de la siguiente manera:
return this.fechaCheckOut.plusDays(cantidadDias);
¿Esta línea constituye un error de lógica respecto al principio de encapsulamiento o inmutabilidad?
¿Modifica el estado real del objeto o solo devuelve un cálculo? Argumente si esto cumple con lo solicitado en el inciso (h).
- 4) Desarrollar una clase llamada Paciente para gestionar información de salud básica. Esta clase debe tener los siguientes atributos: nombre, dni, peso (en kg) y altura (en metros). La clase Paciente debe incluir los siguientes métodos:
- a) Métodos para establecer y obtener el nombre, dni, peso y altura del paciente.
 - b) Un método que calcule y devuelva el Índice de Masa Corporal (IMC) (Fórmula: peso / (altura * altura)).

Estructura de Datos	TRABAJO PRÁCTICO N° 1 - Ciclo 2026 Tema: POO
INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY	

- c) Un método que devuelva un String indicando el estado nutricional según el IMC: "Bajo peso" si el IMC es menor a 18.5, "Normal" si está entre 18.5 y 24.9, "Sobrepeso" si está entre 25 y 29.9, y "Obesidad" si es mayor o igual a 30.

En el programa principal (main) se debe:

- d) Crear una instancia de la clase Paciente.
e) Solicitar al usuario que ingrese el nombre, el dni, el peso y la altura del paciente, validando que peso y altura sean valores numéricos positivos.
f) Utilizar los métodos de la clase Paciente para establecer estos valores.
g) Mostrar el nombre del paciente, el dni, el valor de su IMC (formateado a dos decimales) y su estado nutricional.

Indicaciones:

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

Preguntas sobre el problema

- a) Al diseñar la clase, ¿el IMC debería ser un atributo (ej. double imc;) o solo el resultado de un método de cálculo (ej. calcularIMC())? Tenga en cuenta que inicialmente el peso puede recibir un valor, pero luego se puede invocar al método setter para cambiarlo.
b) Encontrar valores de peso y altura tales que los cálculos de IMC correspondientes sean iguales a 24.9 y a 25.0 (Ayuda: fijar un valor para una de las variables y despejar la otra usando la fórmula de IMC). ¿Qué estado nutricional se obtiene en cada caso? ¿Por qué es crucial probar estos límites en las estructuras condicionales?
c) Un compañero escribe la validación de la altura de una persona en el método main de esta manera:
- ```
if (altura > 0) {
 paciente.setAltura(altura);
}
```

Sin embargo, más tarde descubre que usando los setters de forma directa alguien pudo asignarle una altura de -2 al objeto. ¿Qué falta en el diseño de la clase Paciente para garantizar que sus atributos NUNCA tengan valores negativos sin importar quién use la clase? Argumente sobre cómo deberían ser los métodos setter.

|                                                                                                                               |                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 1 - Ciclo 2026</b><br><br><b>Tema: POO</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                   |

5) Leer atentamente el enunciado presentado a continuación y luego, analizar las soluciones propuestas.

#### Enunciado:

Diseñar una clase llamada Bateria que representa la carga de un dispositivo. La clase debe tener un atributo para el porcentaje de carga (un valor entero entre 0 y 100). Debe permitir cargar la batería (sumando porcentaje, sin superar 100) y consumir batería (restando porcentaje, sin bajar de 0). En el programa principal, se debe crear una batería con 50% de carga, consumir 60 unidades (lo que debería dejarla en 0) y mostrar el estado final.

#### **Solución A**

```
class Bateria {
 public int carga;

 public Bateria(int cargaInicial) {
 if (cargaInicial < 0) cargaInicial = 0;
 if (cargaInicial > 100) cargaInicial = 100;
 this.carga = cargaInicial;
 }

 public void consumir(int cantidad) {
 this.carga = this.carga - cantidad;
 }
}

public class Main {
 public static void main(String[] args) {
 Bateria b = new Bateria(50);

 // Consumir 60
 if (b.carga - 60 < 0) {
 b.carga = 0;
 } else {
 b.carga = b.carga - 60;
 }

 System.out.println("Carga final: " + b.carga + "%");
 }
}
```

#### **Solución B**

```
class Bateria {
 private int carga;
```

|                                                                                                                               |                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 1 - Ciclo 2026</b><br><br><b>Tema: POO</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                   |

```

public Bateria(int cargaInicial) {
 if (cargaInicial < 0) cargaInicial = 0;
 if (cargaInicial > 100) cargaInicial = 100;
 this.carga = cargaInicial;
}

public void consumir(int cantidad) {
 this.carga -= cantidad;
 if (this.carga < 0) {
 this.carga = 0;
 }
}

public int getCarga() {
 return this.carga;
}
}

public class Main {
 public static void main(String[] args) {
 Bateria b = new Bateria(50);

 // Consumir 60
 b.consumir(60);

 System.out.println("Carga final: " + b.getCarga() + "%");
 }
}

```

### **Preguntas sobre el problema**

1. ¿Cuál de las dos soluciones representa mejor los principios de la Programación Orientada a Objetos estudiados? Justifique su respuesta haciendo referencia al concepto de encapsulamiento.
2. Si tuvieras que explicar por qué la Solución B es un mejor diseño de software a un compañero que recién empieza a programar, ¿qué argumentos le darías?
3. El enunciado pide que el valor "no baje de 0". ¿Cuál de las dos soluciones respeta mejor esta restricción de manera robusta? ¿Qué sucede en la *Solución A* si en el futuro otro programador escribe `b.consumir(200)` sin incluir una estructura selectiva `if` en el método `main`? ¿Y qué sucede en la *Solución B* en ese mismo caso?
4. Identificar al menos una ventaja de la *Solución A* y una ventaja de la *Solución B*. Luego, explicar por qué la *Solución B* puede ser más valiosa en un sistema de software real que puede crecer en el futuro.

|                                                                                                                               |                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 1 - Ciclo 2026</b><br><br><b>Tema: POO</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                   |

6) Leer atentamente el enunciado presentado a continuación y luego, analizar la solución propuesta. ¡Cuidado! La solución contiene errores.

Enunciado:

Diseñar una clase TanqueAgua que represente un contenedor de líquido. La clase debe tener los atributos capacidadMaxima (en litros) y cantidadActual (en litros).

Además, debe incluir:

- Un constructor predeterminado que inicialice capacidadMaxima en 100 y cantidadActual en 0.
- Un constructor parametrizado que reciba ambos valores. Si la cantidadActual excede la capacidadMaxima, se debe ajustar la cantidadActual para que sea igual a la capacidad máxima.
- Un método agregarAgua(double litros) que sume los litros a la cantidadActual. Si se excede la capacidad máxima, el tanque debe quedar lleno y mostrar un mensaje de advertencia.
- Un método retirarAgua(double litros) que reste los litros. Si se intenta retirar más agua de la disponible, el tanque debe quedar en 0.
- Un método mostrarEstado() que imprima la cantidad actual y la capacidad máxima.

**Solución propuesta (¡Contiene errores!):**

```
class TanqueAgua {
 public double capacidadMaxima;
 public double cantidadActual;

 public TanqueAgua () {
 this.capacidadMaxima = 100;
 this.cantidadActual = 0;
 }

 public TanqueAgua(double capacidadMaxima, double cantidadActual) {
 this.capacidadMaxima = capacidadMaxima;
 this.cantidadActual = cantidadActual;
 }

 public void agregarAgua(double litros) {
 this.cantidadActual = this.cantidadActual + litros;
 if (this.cantidadActual > this.capacidadMaxima) {
 System.out.println("Advertencia: Se excedio la capacidad.");
 }
 }
}
```



|                                                                                                                               |                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 1 - Ciclo 2026</b><br><br><b>Tema: POO</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                   |

```

public void retirarAgua(double litros) {
 this.cantidadActual = this.cantidadActual - litros;
}

public void mostrarEstado() {
 System.out.println("Actual: " + this.cantidadActual + " / Maxima: " +
this.capacidadMaxima);
}
}

```

**Actividad:**

1. Analizar el código línea por línea e identificar los errores conceptuales y lógicos.
2. Argumentar acerca de por qué constituyen errores. (Ayuda: Hay un error grave de encapsulamiento, un error en el constructor parametrizado, un error de lógica en agregarAgua y otro en retirarAgua).
3. ¿La solución responde completamente a lo solicitado por la consigna? ¿Se cumple la regla de que el tanque "debe quedar lleno" o "debe quedar en 0" en los casos límite?
4. Reescribir la clase TanqueAgua para que la solución sea correcta, respetando los principios de la Programación Orientada a Objetos.